



← Back to student guides

✦ Airflow

Orchestration

3 levels

113 sections



The Complete Guide

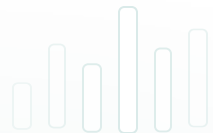
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

DAG

SCHEDULER

EXECUTOR

WORKER



16

sections

36

examples





is repeated for its own sake. where you already know the basics, we go straight to the depth.

Where this picks up

Topic you already use	What this level adds
The five components	Executors compared, the triggerer, and where each one becomes the bottleneck
DAGs and tasks	TaskGroups, DAG factories, and generating DAGs safely from config
Dependencies	<code>chain</code> , <code>cross_downstream</code> , <code>setup</code> / <code>teardown</code> , and label edges
Scheduling	Timetables, cron vs <code>timedelta</code> semantics, <code>@once</code> , and catchup strategy
Retries and timeouts	SLAs, callbacks, <code>on_failure_callback</code> , and alerting that people read
XCom	Custom backends, size limits, and passing data through object storage
Connections and Variables	<code>Params</code> , <code>dag_run.conf</code> , and configuration precedence
Templating	<code>template_fields</code> , custom macros, <code>user_defined_macros</code> , and rendering pitfalls
Sensors	Deferrable operators and the triggerer: why sensors are now a fallback
Branching	Trigger rules in full, <code>latest_only</code> , short-circuiting, and setup/teardown
The UI and CLI	Testing DAGs properly, <code>dag.test()</code> , and CI that blocks a broken DAG
Top-level code	Parse-time budgets, <code>.airflowignore</code> , and measuring the scheduler
new	Dynamic task mapping · datasets · pools and priority · custom operators · CI



Executors, and where each one hurts

Beginner treated the executor as a black box. Choosing it is the single most consequential architectural decision in an Airflow deployment.

Executor	Runs tasks	Parallelism	Use when
<code>SequentialExecutor</code>	In the scheduler, one at a time	None	Never, outside a first tutorial
<code>LocalExecutor</code>	Subprocesses on the scheduler host	One machine	Small teams, one box, simple ops
<code>CeleryExecutor</code>	Long-lived workers via a queue	Horizontal, pre-provisioned	Steady load, many small tasks
<code>KubernetesExecutor</code>	One pod per task	Horizontal, on demand	Variable load, per-task isolation
<code>CeleryKubernetesExecutor</code>	Both, by queue	Both	Mixed workloads

The trade-off that matters is **startup latency versus isolation**:

✓ Celery

- Workers already running: task starts in under a second
- Efficient for thousands of short tasks
- Queues let you route work to specific worker pools
- But: shared environment, and a dependency conflict affects everyone
- But: you pay for idle workers

✗ Kubernetes

- Per-task pod: full isolation, per-task image and resources
- Scales to zero. You pay for what runs
- A task needing 64 GB gets 64 GB without resizing a worker
- But: pod startup is 10–30 seconds, every task



airflow.cfg

Copy

```
[core]
executor = CeleryExecutor
parallelism = 64                # cluster-wide simultaneous task limit
max_active_tasks_per_dag = 16
max_active_runs_per_dag = 3
dag_file_processor_timeout = 60

[celery]
worker_concurrency = 16        # tasks per worker process
```

The three limits interact and people get caught by the tightest one:

Setting	Scope	Symptom when too low
<code>parallelism</code>	The whole cluster	Tasks stuck in <code>queued</code> across every DAG
<code>max_active_tasks_per_dag</code>	One DAG	One DAG runs slowly while others are fine
<code>worker_concurrency</code>	One worker	Queue depth grows; workers are not saturated
Pool slots	A named group	Only tasks in that pool queue up

The **triggerer** is the sixth component, and it exists for deferrable operators. It runs an asyncio event loop that can hold tens of thousands of waiting tasks in one process, which is the point of the next section but one.

► TRY IT

1. Find your executor: `airflow config get-value core executor` .
2. Read `parallelism` , `max_active_tasks_per_dag` , and if you are on Celery, `worker_concurrency` .
3. Create a DAG with twenty parallel `sleep 30` tasks and watch the UI. Count how many run simultaneously.
4. Lower `max_active_tasks` on the DAG to 3 and rerun. Confirm the number matches.



limits binds first is what makes tasks are stuck in queue a five minute diagnosis.

Dynamic task mapping

Beginner's DAGs had a fixed task list. Real pipelines often need "one task per file that landed", and the count is unknown until run time.

PYTHON

Copy

```
@task
def list_files() → list[str]:
    hook = S3Hook(aws_conn_id="aws_default")
    return hook.list_keys(bucket_name="raw", prefix="events/{{ ds }}/")

@task
def process(key: str) → int:
    ...
    return rows

@task
def summarise(counts: list[int]):
    print(f"Total rows: {sum(counts)}")

counts = process.expand(key=list_files())    # N parallel task instances
summarise(counts)
```

`.expand()` creates one **mapped task instance** per element, at run time, and the UI groups them under a single task with an index. `summarise` receives the list of all results. That collection is automatic.

Three forms cover almost everything:

PYTHON

Copy



```
# Several arguments vary together, zipped
process.expand(key=keys, region=regions)          # cross product!
process.expand_kwargs([{"key": "a", "region": "eu"}, {"key": "b", "region": "us"}])

# Some arguments fixed, some varying
process.partial(bucket="raw", retries=5).expand(key=list_files())
```

expand with two arguments is a cross product, not a zip

`process.expand(key=[a, b], region=[eu, us])` creates **four** task instances, not two. If you want pairs, use `expand_kwargs` with a list of dicts, or zip the lists in an upstream task. Discovering this by watching 400 tasks appear instead of 20 is a memorable afternoon.

Mapping classic operators works too:

PYTHON

Copy

```
BashOperator.partial(task_id="process").expand(
    bash_command=["echo one", "echo two", "echo three"],
)
```

The limits are real, so know them:

airflow.cfg

Copy

```
[core]
max_map_length = 1024          # the cap on mapped instances per task
```

Mapping is not a substitute for a batch job

Ten thousand mapped tasks means ten thousand scheduler decisions, ten thousand database rows, and, on Kubernetes, ten thousand pod launches. If the per-item work is



► TRY IT

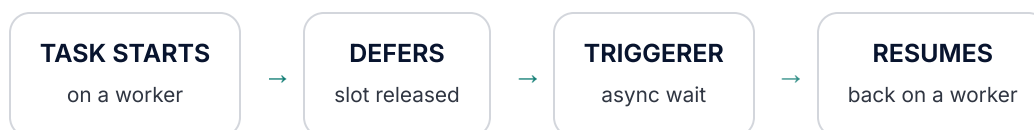
1. Write a DAG where one task returns a list of five strings and a second maps over them.
2. Open the mapped task in the UI and browse the individual indexed instances.
3. Make instance 3 fail, then clear only that one and watch it rerun alone.
4. Now try `expand` with two lists of three elements each and count the resulting instances.

Expected result – five indexed instances you can retry individually, and nine instances from two lists of three. That second result is the cross-product surprise, and it is much better to meet it here than in production.

Deferrable operators and the triggerer

Beginner used sensors and learned that `poke` mode wastes worker slots. Deferrable operators remove the problem entirely.

A deferrable operator starts, discovers it must wait, and **defers**: it releases its worker slot, handing a lightweight `Trigger` to the triggerer process. The triggerer runs thousands of these in one asyncio loop. When the condition is met, the task is rescheduled onto a worker to finish.



PYTHON

Copy

```
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.sensors.time_delta import TimeDeltaSensorAsync

wait = S3KeySensor(
    task_id="wait_for_file",
```



```
poke_interval=60,  
timeout=60 * 60 * 4,  
)
```

Approach	Worker slots while waiting	Scales to
Sensor, <code>poke</code> mode	One per sensor, continuously	Tens
Sensor, <code>reschedule</code> mode	Zero between checks, one per check	Hundreds
Deferrable , <code>deferrable=True</code>	Zero	Tens of thousands

Many operators are deferrable, not just sensors. `deferrable=True` on a Spark submit, a BigQuery job, or an EMR step means Airflow does not hold a worker while an external system works.

Writing your own is a class and a trigger:

PYTHON

Copy

```
from airflow.triggers.base import BaseTrigger, TriggerEvent  
  
class TableReadyTrigger(BaseTrigger):  
    def __init__(self, table: str, day: str):  
        super().__init__()  
        self.table, self.day = table, day  
  
    def serialize(self):  
        return ("mypkg.triggers.TableReadyTrigger",  
                {"table": self.table, "day": self.day})  
  
    async def run(self):  
        while True:  
            if await self._rows_exist():  
                yield TriggerEvent({"status": "ready"})  
            return  
            await asyncio.sleep(60)
```



```
class WaitForTable(BaseOperator):
    def execute(self, context):
        self.defer(
            trigger=TableReadyTrigger(self.table, context["ds"]),
            method_name="execute_complete",
        )

    def execute_complete(self, context, event=None):
        return event["status"]
```

Deferrable operators need a running triggerer

If no triggerer process exists, deferred tasks sit forever in `deferred` state and nothing tells you loudly. Confirm it is running before adopting them, and treat it as a required component rather than an optional extra. Also note that trigger code must be **async**: a blocking call inside a trigger stalls every other deferred task in that process.

► TRY IT

1. Confirm the triggerer is running: `docker compose ps`, or check the **Admin → Health** page.
2. Write a DAG with twenty `TimeDeltaSensorAsync` tasks waiting five minutes each.
3. Watch the UI: all twenty enter `deferred` and no worker slots are consumed.
4. Now do the same with plain sensors in `poke` mode and compare worker occupancy.

Expected result – twenty tasks waiting with zero workers busy, against twenty tasks saturating your worker pool. That contrast is the entire argument for deferrable operators, and it is visible in one screen.

Datasets and data-aware scheduling

Beginner scheduled on time. Sometimes the correct trigger is not "it is 2am" but "the upstream table was updated", and chaining DAGs with `ExternalTaskSensor` is fragile because



PYTHON

Copy

```
from airflow.datasets import Dataset

events_table = Dataset("postgres://warehouse/analytics.daily_events")
features = Dataset("s3://ml/features/latest.parquet")

# Producer: declares that a task updates a dataset
@dag(dag_id="build_events", schedule="0 2 * * *", start_date=..., catchup=False)
def build_events():
    @task(outlets=[events_table])
    def load():
        ...
    load()

# Consumer: scheduled by the dataset, not by the clock
@dag(dag_id="build_features", schedule=[events_table], start_date=..., catchup=False)
def build_features():
    @task(outlets=[features])
    def featurize():
        ...
    featurize()

# Waits for BOTH upstream datasets
@dag(dag_id="train_model", schedule=[events_table, features], ...)
def train_model():
    ...
```

The consumer runs when every dataset in its `schedule` list has been updated since its last run. No sensor, no polling, no shared schedule assumption, and the **Datasets** view in the UI draws the resulting cross-DAG graph.

Approach	Coupling	Cost
<code>ExternalTaskSensor</code>	Both DAGs must share an interval	A waiting task, and brittle date logic
<code>TriggerDagRunOperator</code>	The producer must know its consumers	Producer changes on every new consumer



Datasets	Neither knows the other, they share a URI	None while waiting
----------	---	--------------------

The dataset URI is just a string

Airflow does not check the resource exists or was written. The URI is an identifier the two DAGs agree on. That makes it flexible and means you should establish a naming convention early: `scheme://system/schema.table` is a reasonable one. Inconsistent URIs mean two DAGs that `look` connected in the code and are not connected at all.

You can also combine time and data conditions, and express logic across them:

PYTHON

Copy

```
from airflow.timetables.datasets import DatasetOrTimeSchedule
from airflow.timetables.trigger import CronTriggerTimetable

@dag(
    schedule=DatasetOrTimeSchedule(
        timetable=CronTriggerTimetable("0 6 * * *", timezone="UTC"),
        datasets=(events_table & features),      # AND; use | for OR
    ),
    ...
)
```

► TRY IT

1. Write two DAGs: a producer with `outlets=[Dataset("x://demo")]` on a schedule, and a consumer with `schedule=[Dataset("x://demo")]`.
2. Trigger the producer manually and watch the consumer start on its own.
3. Open the **Datasets** view and read the graph it drew.
4. Add a second dataset to the consumer's schedule and confirm it now waits for both.

Expected result – a consumer DAG that starts because a dataset was updated, with no sensor and no shared schedule. The Datasets view drawing your cross-DAG dependency graph is the part that changes how you design pipelines.



A DAG with sixty tasks is unreadable in the Graph view. **TaskGroups** collapse related tasks into one expandable node.

PYTHON

Copy

```
from airflow.utils.task_group import TaskGroup

with DAG(...) as dag:
    start = EmptyOperator(task_id="start")

    with TaskGroup(group_id="extract") as extract:
        for source in ["orders", "users", "events"]:
            BashOperator(task_id=source, bash_command=f"python extract.py --src {source}")

    with TaskGroup(group_id="validate") as validate:
        schema = EmptyOperator(task_id="schema")
        rows = EmptyOperator(task_id="row_counts")
        schema >> rows

    start >> extract >> validate
```

TaskGroups are organisational. They create no runtime overhead. The `task_id` becomes `extract.orders`, which matters when you target a task from the CLI. The decorator form is neater in TaskFlow:

PYTHON

Copy

```
from airflow.decorators import task_group

@task_group(group_id="quality_checks")
def quality_checks(table: str):
    @task
    def check_nulls(t: str): ...

    @task
    def check_freshness(t: str): ...

    check_nulls(table) >> check_freshness(table)
```



dags/table_pipelines.py

Copy

```
import yaml
from pathlib import Path

CONFIG = Path(__file__).parent / "config" / "tables.yaml"

def build_dag(name: str, cfg: dict):
    @dag(
        dag_id=f"load_{name}",
        start_date=datetime(2024, 1, 1),
        schedule=cfg["schedule"],
        catchup=False,
        tags=["generated", cfg["domain"]],
    )
    def _dag():
        SQLExecuteQueryOperator(
            task_id="load",
            conn_id=cfg["conn_id"],
            sql=f"sql/{name}.sql",
        )
    return _dag()

# Reading a local YAML at parse time is cheap and acceptable.
for name, cfg in yaml.safe_load(CONFIG.read_text()).items():
    globals()[f"load_{name}"] = build_dag(name, cfg)
```

A factory must read config from a file, not from a database or an API

This code runs on **every parse**. A local YAML read is microseconds and fine. A database query or an HTTP call to build the DAG list means every parse cycle depends on that system being up and fast, and when it is slow, your entire scheduler is slow. If the config lives in a database, sync it to a file on a schedule and let the factory read the file.

► TRY IT

1. Take a DAG with several related tasks and wrap them in two TaskGroups. Confirm the Graph view collapses them.



4. Add a `time.sleep(2)` to the factory loop and watch the parse duration in **Admin → DAG Processing**.

Expected result – a readable Graph view, dotted task ids you can address from the CLI, three generated DAGs, and a measurable parse cost from one careless line in the factory. That last measurement is why the warning above matters.

Pools, priority, and concurrency control

Beginner set `max_active_runs`. That protects one DAG. **Pools** protect a shared resource across every DAG.

The classic problem: a legacy database accepts five concurrent connections. Ten DAGs each with a task hitting it will open forty and take it down.

BASH

Copy

```
airflow pools set legacy_db 5 "Legacy Oracle: max 5 concurrent connections"
airflow pools set gpu 2 "GPU nodes"
airflow pools list
```

PYTHON

Copy

```
SQLExecuteQueryOperator(
    task_id="legacy_extract",
    conn_id="legacy_oracle",
    sql="sql/extract.sql",
    pool="legacy_db",
    pool_slots=1,           # a heavy task can take several slots
    priority_weight=10,     # higher wins when the pool is contended
)
```



parallelism	Cluster	The whole installation
max_active_tasks	One DAG	Other DAGs from one greedy DAG
max_active_runs	One DAG	A table from concurrent writers
Pool	A named resource, across all DAGs	An external system
pool_slots	One task	Lets a heavy task count as several
priority_weight	One task	Which queued task goes first
queue	Celery routing	Sends work to specific workers

`priority_weight` has a detail worth knowing: by default it is `downstream` , meaning a task's effective weight includes its downstream tasks, so a task blocking a long chain naturally outranks a leaf. You can set `weight_rule="absolute"` to use the number literally.

A pool is a promise to an external system

The number in a pool should come from the thing you are protecting: the connection limit, the API rate limit, the number of GPUs. Setting pool sizes by guessing means either an unprotected system or wasted capacity. Write the reason in the pool's description field. The next person will need it.

► TRY IT

1. Create a pool with two slots. Put six `sleep 20` tasks in it.
2. Run the DAG and confirm only two run at a time regardless of your cluster's capacity.
3. Give two of the tasks `priority_weight=100` and rerun. Confirm they go first.
4. Give one task `pool_slots=2` and confirm it runs alone.

Expected result – a hard concurrency ceiling that ignores cluster capacity, priority that visibly reorders the queue, and one task consuming the whole pool. Those three behaviours are all you need to protect any shared system.



Beginner's DAG failed silently unless someone looked. Alerting is the difference between a pipeline you own and one that owns you.

PYTHON

Copy

```
def alert_failure(context):
    ti = context["task_instance"]
    send_slack(
        channel="#data-alerts",
        text=(
            f":red_circle: *{ti.dag_id}.{ti.task_id}* failed\n"
            f"Run: {context['run_id']} Logical date: {context['ds']}\n"
            f"Try {ti.try_number} of {ti.max_tries + 1}\n"
            f"<{ti.log_url}|Logs>"
        ),
    )

def alert_sla_miss(dag, task_list, blocking_task_list, slas, blocking_tis):
    send_slack(channel="#data-alerts", text=f":hourglass: SLA missed: {task_list}")

default_args = {
    "retries": 3,
    "on_failure_callback": alert_failure,
    "sla": timedelta(hours=1),          # per task: expected completion window
}

@dag(
    ...,
    default_args=default_args,
    sla_miss_callback=alert_sla_miss,
    on_failure_callback=alert_failure,  # DAG-level: the run failed
)
```

Callback	Fires when
<code>on_failure_callback</code>	The task (or DAG run) failed after retries
<code>on_success_callback</code>	It succeeded



<code>on_retry_callback</code>	A retry is about to happen
<code>on_execute_callback</code>	Just before execution
<code>on_skipped_callback</code>	It was skipped
<code>sla_miss_callback</code>	A task exceeded its <code>sla</code> window

An **SLA** in Airflow means "this task should complete within X of the DAG run's start". It does not stop or fail anything. It records a miss and fires the callback. That is a genuine limitation worth knowing: for a hard stop you need `execution_timeout`.

Alert on the right thing, or nobody will read your alerts

A callback on every task in every DAG produces a channel nobody looks at, which is worse than no alerting. Alert on **the DAG run failing** and on **SLA misses for pipelines with real consumers**. Route the noisy rest to a dashboard. Always include the log URL: an alert you cannot act on from your phone is an alert you will act on tomorrow.

The other half of alerting is making failures self-describing:

PYTHON

Copy

```
@dag(
    ...,
    doc_md="""
    ### Daily events pipeline
    **Owner:** data-platform · **Consumers:** exec dashboard, ML features
    **On failure:** safe to clear and rerun; idempotent.
    **Escalate if:** not green by 07:00 UTC.
    """,
)
```

That renders in the UI, so whoever is on call at 3am has the runbook in front of them.

► TRY IT



3. Add `"sla": timedelta(seconds=30)` to a task that sleeps for sixty and watch the SLA miss appear under **Browse → SLA Misses**.

4. Add a `doc_md` to the DAG and find it rendered in the UI.

Expected result – a single actionable alert rather than one per retry, a recorded SLA miss that did not stop anything, and a runbook visible in the UI. Step two is the detail people get wrong: callbacks fire on final failure, not on every attempt.

Custom operators and hooks

You will hit work that no provider covers. Writing an operator is short, and it is how a team stops copy-pasting the same forty lines into nine DAGs.

plugins/operators/quality_check.py

Copy

```
from airflow.models import BaseOperator
from airflow.utils.decorators import apply_defaults
from airflow.exceptions import AirflowFailException

class RowCountCheckOperator(BaseOperator):
    """Fails if a table's row count for the interval is below a threshold."""

    template_fields = ("table", "day", "sql")  # ← makes {{ ds }} work
    ui_color = "#e8f5e9"

    def __init__(self, *, conn_id: str, table: str, day: str,
                 min_rows: int = 1, **kwargs):
        super().__init__(**kwargs)
        self.conn_id, self.table, self.day, self.min_rows = conn_id, table, day, min_row
        self.sql = f"SELECT count(*) FROM {table} WHERE day = %s"

    def execute(self, context) → int:
        hook = PostgresHook(postgres_conn_id=self.conn_id)
        count = hook.get_first(self.sql, parameters=(self.day,))[0]
```



```
# will not fix itself by trying again.
raise AirflowFailException(
    f"{self.table} has {count} rows for {self.day}, expected ≥ {self.min_ro
)
return count
```

PYTHON

Copy

```
RowCountCheckOperator(
    task_id="check_events",
    conn_id="warehouse",
    table="staging.events",
    day="{{ ds }}",          # templated because `day` is in template_fields
    min_rows=1000,
)
```

Four details in there are the lesson:

template_fields is what makes `{{ ds }}` work in your operator's arguments. Forget it and you get the literal string, a common bug in first custom operators.

self.log is the task logger, so your messages appear in the task's log in the UI rather than in the worker's stdout.

AirflowFailException fails without consuming retries, which is correct for a data-quality failure: retrying will not add the missing rows. **AirflowSkipException** is its sibling, marking the task skipped.

Returning a value pushes it to XCom automatically.

A **hook** wraps a connection. Write one when you have a system with no provider:

plugins/hooks/internal_api.py

Copy

```
from airflow.hooks.base import BaseHook
import requests

class InternalApiHook(BaseHook):
```



```
def __init__(self, internal_conn_id: str = default_conn_name):
    super().__init__()
    self.internal_conn_id = internal_conn_id

def get_conn(self) → requests.Session:
    conn = self.get_connection(self.internal_conn_id) # credentials from Airflow
    session = requests.Session()
    session.headers["Authorization"] = f"Bearer {conn.password}"
    session.base_url = conn.host
    return session

def fetch(self, path: str) → dict:
    return self.get_conn().get(f"{self.get_connection(self.internal_conn_id).host}{p
```

Write the operator when the pattern appears the third time

Two copies of similar task code is fine. The third is the signal: extract an operator, put it in a package the whole team installs, and version it. A shared operator is also where you enforce standards: the row-count check that every table load must pass is far more likely to happen if it is one line rather than forty.

► TRY IT

1. Write a small custom operator with one templated field and use it in a DAG.
2. Check the **Rendered Template** tab and confirm the field resolved.
3. Remove that field from `template_fields` and rerun to see the literal `{{ ds }}`.
4. Raise `AirflowFailException` from it and confirm the task fails

Expected result – without
using its retries.

Expected result – a working custom operator, then the literal-template bug, then a data failure that correctly refuses to retry. Those three results cover the operator-authoring mistakes people make.



Beginner's DAGs did the same thing every run. **Params** let a human parameterise a manual trigger, with validation and a form in the UI.

PYTHON

Copy

```
from airflow.models.param import Param

@dag(
    dag_id="reprocess_range",
    schedule=None,
    start_date=datetime(2024, 1, 1),
    params={
        "start_date": Param("2024-05-01", type="string", format="date",
                             title="Start date"),
        "tables": Param(["events"], type="array",
                        description="Which tables to reprocess"),
        "full_refresh": Param(False, type="boolean"),
        "batch_size": Param(1000, type="integer", minimum=1, maximum=100_000),
    },
)
def reprocess_range():

    @task
    def report(**context):
        p = context["params"]
        print(f"Reprocessing {p['tables']} from {p['start_date']}, full={p['full_refresh']}")

    BashOperator(
        task_id="run",
        bash_command="python reprocess.py --start {{ params.start_date }} "
                     "--batch {{ params.batch_size }}",
    )
```

Triggering with **Trigger DAG w/ config** gives you a generated form with the right widget for each type, and validation before the run starts.

BASH

Copy



Mechanism	Set by	Read as	Scope
Variable	An operator, persistently	<code>Variable.get()</code> / <code>{{ var.value.x }}</code>	Global
params	The DAG author, with a default	<code>{{ params.x }}</code> / <code>context["params"]</code>	Per run, validated
dag_run.conf	Whoever triggers the run	<code>{{ dag_run.conf["x"] }}</code>	Per run, unvalidated

params over raw conf

`dag_run.conf` is an untyped dict, so a typo in a key produces a `KeyError` halfway through the run, at 3am, in a task nobody is watching. `params` gives you defaults, types, bounds, and a UI form, and it fails before the run starts. Use `conf` only for programmatic triggers where the caller is also code you control.

► TRY IT

1. Add three params of different types to a manual-only DAG.
2. Use **Trigger DAG w/ config** and note the generated form and its validation.
3. Try to submit a value outside a numeric bound and read the rejection.
4. Trigger the same DAG from the CLI with `--conf` and confirm the values arrive.

Expected result – a validated form generated from your parameter declarations, and a rejection before any task ran. That pre-flight validation is the difference between `params` and a bare `conf` dict.

Testing DAGs



Layer one: does it import, and is the structure sane? This catches the majority of real breakages and runs in seconds.

tests/test_dag_integrity.py

Copy

```
import pytest
from airflow.models import DagBag

@pytest.fixture(scope="session")
def dagbag():
    return DagBag(dag_folder="dags/", include_examples=False)

def test_no_import_errors(dagbag):
    assert not dagbag.import_errors, f"Import errors: {dagbag.import_errors}"

def test_every_dag_has_required_metadata(dagbag):
    for dag_id, dag in dagbag.dags.items():
        assert dag.tags, f"{dag_id} has no tags"
        assert dag.default_args.get("retries", 0) ≥ 1, f"{dag_id} has no retries"
        assert dag.catchup is False, f"{dag_id} has catchup enabled"
        assert dag.dagrun_timeout, f"{dag_id} has no dagrun_timeout"
        assert dag.doc_md, f"{dag_id} has no documentation"

def test_no_cycles(dagbag):
    for dag in dagbag.dags.values():
        dag.test_cycle()

def test_parse_time_is_reasonable(dagbag):
    assert dagbag.dagbag_stats
    slow = [s for s in dagbag.dagbag_stats if s.duration.total_seconds() > 2]
    assert not slow, f"Slow-parsing files: {[s.file for s in slow]}"
```

Layer two: unit-test the logic, by extracting it from the operator.

PYTHON

Copy

```
# The business logic lives in a plain function...
def classify_load(row_count: int, threshold: int = 100_000) → str:
    return "full_reload" if row_count > threshold else "incremental_load"
```



```
assert classify_load(500) == "incremental_load"
```

Layer three: run the DAG. `dag.test()` executes a complete run in-process, which is the closest you get to an integration test.

tests/test_pipeline_run.py

Copy

```
from airflow.models import DagBag

def test_pipeline_runs_end_to_end():
    dag = DagBag(dag_folder="dags/", include_examples=False).get_dag("daily_events_pipeline")
    dag.test(execution_date=datetime(2024, 5, 1))    # raises on failure
```

.github/workflows/dags.yml

Copy

```
name: DAGs
on: [pull_request]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11', cache: pip }
      - run: pip install -r requirements.txt
      - run: ruff check dags/ plugins/
      - run: pytest tests/ -v                # integrity + unit tests
      - run: python -c "from airflow.models import DagBag; \
        b=DagBag('dags/', include_examples=False); \
        assert not b.import_errors, b.import_errors"
```

The integrity test is the one that earns its keep

A single import error in one DAG file can stop the scheduler processing it, and if the error is in a shared module it can affect many. The `DagBag` test takes seconds, needs no database, and catches the most common and most damaging class of breakage. If you add only one test, add that one.



1. write the integrity test and run it. Fix anything it flags in your own DAGs.
2. Add the metadata test asserting retries, tags, and `dagrun_timeout` . Watch it fail on a real DAG, then fix the DAG.
3. Break a DAG file with a syntax error and confirm the test catches it.
4. Run `dag.test()` on a small DAG and watch it execute in your terminal.

Expected result – a test suite that enforces your own standards, failing on real DAGs you had thought were fine. That metadata test is the cheapest way to make a convention stick across a team.

Timetables and scheduling beyond cron

Cron cannot express "every weekday at 6am, but not on public holidays" or "the first business day of each month". A **Timetable** can.

plugins/timetables/business_days.py

Copy

```
from airflow.timetables.base import DagRunInfo, DataInterval, TimeRestriction, Timetable
from pendulum import DateTime, Time

class BusinessDayTimetable(Timetable):
    """Runs at 06:00 on weekdays only, covering the previous business day."""

    def infer_manual_data_interval(self, run_after: DateTime) → DataInterval:
        start = run_after.subtract(days=1).start_of("day")
        return DataInterval(start=start, end=start.add(days=1))

    def next_dagrun_info(self, *, last_automated_data_interval, restriction):
        if last_automated_data_interval is None:
            start = restriction.earliest
            if start is None:
                return None
        else:
            start = last_automated_data_interval.end

        # Skip forward to the next weekday
```



```
if restriction.latest is not None and start > restriction.latest:
    return None

return DagRunInfo.interval(
    start=start,
    end=start.add(days=1),
)
```

PYTHON

Copy

```
@dag(schedule=BusinessDayTimetable(), start_date=datetime(2024, 1, 1), catchup=False)
def weekday_report():
    ...
```

Before writing one, check the built-ins. They cover more than people expect:

Timetable	Expresses
<code>CronTriggerTimetable</code>	Cron, but fires at the time rather than at interval end
<code>DeltaDataIntervalTimetable</code>	A <code>timedelta</code> , aligned to the previous run
<code>DatasetOrTimeSchedule</code>	Datasets or a schedule, whichever comes first
<code>EventsTimetable</code>	An explicit list of datetimes: ideal for irregular calendars

`CronTriggerTimetable` deserves a note because it resolves a real confusion: a plain cron schedule fires **after** the interval it covers, while `CronTriggerTimetable("0 6 * * *")` fires at 06:00 and treats that instant as the logical date. If you have ever wanted "run at 6am and call it today's run", that is the one.

PYTHON

Copy

```
from airflow.timetables.trigger import CronTriggerTimetable

@dag(schedule=CronTriggerTimetable("0 6 * * *", timezone="Europe/Berlin"), ...)
```



`CronTriggerTimetable` and `DAG(timezone=...)` both accept a real timezone, and Airflow handles the daylight-saving transitions. "The report was an hour late every summer" is a real bug people spend a long time not finding.

► TRY IT

1. Switch a DAG from `schedule="0 6 * * *"` to `CronTriggerTimetable("0 6 * * *")` and compare the logical dates of the resulting runs.
2. Use `EventsTimetable` with three explicit datetimes and confirm only those runs are created.
3. Set a non-UTC timezone and check the **Next Run** column in the UI.
4. Write a minimal custom timetable that skips weekends and confirm no Saturday or Sunday runs appear.

Expected result – the same cron string producing different logical dates under the two timetables, which is exactly the confusion `CronTriggerTimetable` exists to remove. Step one is worth doing even if you never write a custom timetable.

XCom backends and passing real data

Beginner's rule was "XCom for metadata, storage for data". At this level you can make that automatic with a **custom XCom backend**, so large returns go to object storage transparently.

plugins/xcom/s3_backend.py

Copy

```
import json, uuid
from typing import Any
from airflow.models.xcom import BaseXCom
from airflow.providers.amazon.aws.hooks.s3 import S3Hook

BUCKET = "airflow-xcom"
THRESHOLD = 8 * 1024 # anything larger goes to S3
```



```
payload = json.dumps(value).encode()
if len(payload) < THRESHOLD:
    return BaseXCom.serialize_value(value)

key = f"xcom/{uuid.uuid4()}.json"
S3Hook().load_bytes(payload, key=key, bucket_name=BUCKET, replace=True)
return BaseXCom.serialize_value(f"s3://{BUCKET}/{key}")

@staticmethod
def deserialize_value(result):
    value = BaseXCom.deserialize_value(result)
    if isinstance(value, str) and value.startswith(f"s3://{BUCKET}/xcom/"):
        key = value.split(f"{BUCKET}/", 1)[1]
        return json.loads(S3Hook().read_key(key, BUCKET))
    return value
```

airflow.cfg

Copy

```
[core]
xcom_backend = plugins.xcom.s3_backend.S3XComBackend
```

Small values stay in the database and remain visible in the UI; large ones move to S3 with only a pointer stored. DAG code is unchanged.

Approach	Where data lives	UI visibility	Cleanup
Default XCom	Metadata database	Full	airflow db clean
Custom backend	Object storage, pointer in the DB	The pointer only	Your responsibility
Explicit paths in your tasks	Object storage	The path	Your pipeline's

A custom backend makes cleanup your problem



Set an expiry on the XCom prefix. A few days is usually plenty, since XCom values are only read by the same DAG run.

The alternative, and often the better one, is explicitness:

PYTHON

Copy

```
@task
def extract() → str:
    df = fetch_data()
    path = f"s3://staging/events/{{{{ ds }}}}/raw.parquet"
    df.to_parquet(path)
    return path                                # a path, deliberately

@task
def transform(path: str) → str:
    df = pd.read_parquet(path)
    ...
```

The explicit version is more code, and it makes the data flow visible in the DAG, which reviewers appreciate. A backend is better when many DAGs would otherwise each reimplement the same pattern.

► TRY IT

1. Return a 2 MB object from a task with the default backend and time the run. Look at the XCom page.
2. Check the metadata database size before and after.
3. Rewrite it to write a file and return the path. Compare the run time.
4. If you have object storage available, implement the backend above and confirm small values still appear in the UI.

Expected result – a visibly slow run and a bloated XCom table, then a fast one passing a path. Measuring the database growth in step two is what makes the "metadata, not data" rule concrete at this level.



A DAG that provisions a cluster, uses it, and tears it down has a problem: if the middle task fails, the teardown must still run, and `TriggerRule.ALL_DONE` on a cleanup task is a blunt approximation.

Airflow has first-class **setup and teardown** tasks:

PYTHON

Copy

```
@task
def create_cluster() → str:
    return provision_emr()          # returns a cluster id

@task
def run_job(cluster_id: str):
    submit_step(cluster_id)

@task
def destroy_cluster(cluster_id: str):
    terminate_emr(cluster_id)

cluster = create_cluster()
job = run_job(cluster)
teardown = destroy_cluster(cluster)

cluster >> job >> teardown
cluster.as_setup() >> teardown.as_teardown()    # declares the relationship
```

Or more concisely with the decorator arguments:

PYTHON

Copy

```
@task(multiple_outputs=False)
def create_cluster() → str: ...

@task
def destroy_cluster(cluster_id: str): ...
```



Property	<code>ALL_DONE</code> cleanup task	Setup/teardown
Runs after failure	Yes	Yes
Its own failure fails the run	Yes	No by default
Excluded from a clear of the main tasks	No	Yes: teardown reruns as needed
Reflects intent in the UI	No	Yes, drawn distinctly
Skipped when nothing needed it	No	Yes

The behaviour that matters: a **teardown failure does not fail the DAG run** unless you ask it to. That is usually right. If the job succeeded and only the cluster termination was flaky, the pipeline's output is still valid, and you want an alert rather than a red run that masks a success.

Use setup/teardown for anything you pay for by the minute

Ephemeral clusters, GPU nodes, database connections, temporary tables, lock files. The pattern's value is that a failure in the middle cannot leave the expensive resource running, which is a cost bug that only shows up on next month's invoice.

► TRY IT

1. Build a three-task DAG: setup writes a file, work reads it, teardown deletes it.
2. Wire it with `as_setup()` and `as_teardown()` and confirm the UI draws them differently.
3. Make the middle task fail. Confirm teardown still runs and the file is gone.
4. Now make teardown itself fail and check whether the DAG run is red.

Expected result – a teardown that runs after a failure, and a DAG run that stays green when only the teardown failed. That second behaviour is the difference from an `ALL_DONE` cleanup task, and it is usually what you want.



Everything from this level in one DAG. Nothing here is new. Read it as a whole and you should be able to justify every line.

dags/events_platform.py

Copy

```
"""
### Events platform pipeline
**Owner:** data-platform · **Consumers:** exec dashboard, ML features
**On failure:** idempotent – safe to clear and rerun.
**Escalate if:** not green by 07:00 UTC.
"""
from datetime import datetime, timedelta

from airflow.datasets import Dataset
from airflow.decorators import dag, task, task_group
from airflow.models.param import Param
from airflow.operators.empty import EmptyOperator
from airflow.providers.amazon.aws.hooks.s3 import S3Hook
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
from airflow.timetables.trigger import CronTriggerTimetable
from airflow.utils.trigger_rule import TriggerRule

EVENTS_TABLE = Dataset("postgres://warehouse/analytics.daily_events")

def alert(context):
    ti = context["task_instance"]
    print(f"ALERT {ti.dag_id}.{ti.task_id} failed – {ti.log_url}")

default_args = {
    "owner": "data-platform",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "execution_timeout": timedelta(minutes=30),
    "on_failure_callback": alert,
    "sla": timedelta(hours=2),
}

@dag(
    dag_id="events_platform",
```



```
catchup=False,
default_args=default_args,
dagrun_timeout=timedelta(hours=4),
max_active_runs=1,                      # one writer per table
max_active_tasks=12,                    # do not starve other DAGs
tags=["production", "events"],
doc_md=__doc__,
params={
    "full_refresh": Param(False, type="boolean", title="Full refresh"),
    "batch_size": Param(5000, type="integer", minimum=100, maximum=50_000),
},
on_failure_callback=alert,
)

def events_platform():

    start = EmptyOperator(task_id="start")

    # Deferrable: releases the worker slot entirely while waiting.
    wait = S3KeySensor(
        task_id="wait_for_landing",
        bucket_name="raw-data",
        bucket_key="events/{{ ds }}/_SUCCESS",
        deferrable=True,
        poke_interval=60,
        timeout=60 * 60 * 3,
    )

    @task
    def list_partitions() → list[str]:
        """Cheap listing at run time – never at parse time."""
        return S3Hook().list_keys(bucket_name="raw-data", prefix="events/{{ ds }}/part-")

    @task(pool="warehouse_writes", pool_slots=1, priority_weight=10)
    def load_partition(key: str) → int:
        """Pooled: the warehouse accepts a bounded number of writers."""
        return copy_into_staging(key)

    @task_group(group_id="quality")
    def quality(counts: list[int]):
        @task
        def check_rows(c: list[int]):
            if sum(c) == 0:
                raise ValueError("No rows landed for {{ ds }}")
            return sum(c)
```



```
...

check_rows(counts) >> check_freshness()

# Idempotent SQL: DELETE the partition, then INSERT. Marks the dataset.
rollup = SQLExecuteQueryOperator(
    task_id="rollup",
    conn_id="warehouse",
    sql="sql/daily_rollup.sql",
    params={"lookback_days": 7},
    outlets=[EVENTS_TABLE],          # downstream DAGs are scheduled by this
)

cleanup = EmptyOperator(task_id="cleanup", trigger_rule=TriggerRule.ALL_DONE)
end = EmptyOperator(task_id="end")

counts = load_partition.expand(key=list_partitions())    # N mapped instances
start >> wait >> counts
quality(counts) >> rollup >> cleanup >> end

events_platform()
```

dags/features_platform.py

Copy

```
"""Scheduled by data, not by the clock."""
from airflow.datasets import Dataset

EVENTS_TABLE = Dataset("postgres://warehouse/analytics.daily_events")

@dag(dag_id="features_platform", schedule=[EVENTS_TABLE], start_date=..., catchup=False)
def features_platform():
    ...
```

Twelve decisions in there are the whole lesson of this level:

Decision	Section
<code>CronTriggerTimetable</code> with an explicit timezone	Timetables
<code>deferrable=True</code> on the sensor	Deferrable operators



<code>.expand()</code> over a run-time partition list	Dynamic task mapping
<code>pool</code> and <code>priority_weight</code> on the writing task	Pools and priority
<code>max_active_runs=1</code> plus <code>max_active_tasks=12</code>	Executors and limits
<code>outlets=[...]</code> making a downstream DAG data-scheduled	Datasets
A <code>@task_group</code> for the quality checks	TaskGroups
<code>on_failure_callback</code> and an <code>sla</code> in <code>default_args</code>	SLAs and callbacks
<code>params</code> with types and bounds	Params
<code>doc_md</code> carrying the runbook	SLAs and callbacks
Idempotent templated SQL in a file	(Beginner) Templating
No top-level listing, queries, or imports	(Beginner) Top-level code

► TRY IT: THE ONE THAT MATTERS

1. Rebuild a real pipeline of yours against this template and get it green.
2. Verify each mechanism actively: confirm the deferred sensor holds no worker slot, confirm the pool caps concurrency, confirm the downstream dataset DAG starts on its own, confirm a mapped instance can be retried alone.
3. Add the integrity test suite and make it pass, including the assertions about retries, tags, and `dagrun_timeout`.
4. Check the parse duration is under a second, then backfill three days with `max_active_runs=1`.

Expected result – a pipeline that waits without cost, respects a shared resource, triggers its consumer by data, and is retryable at the individual-partition level. Those four properties are what separate a DAG that works from one a platform team is happy to host.



You can choose an executor deliberately, map tasks dynamically at run time, replace sensors with deferrable operators, schedule on data rather than time, organise large DAGs with TaskGroups and factories, protect shared systems with pools and priority, alert in a way people act on, write custom operators and hooks, parameterise runs with validation, test DAGs in CI, express schedules cron cannot, move large XCom values out of the database, and manage expensive resources with setup and teardown.

Can you...	
Give the Celery/Kubernetes trade-off in one sentence?	Startup latency versus per-task isolation
Name the three concurrency limits and their scopes?	<code>parallelism</code> , <code>max_active_tasks</code> , <code>worker_concurrency</code>
Say what <code>expand</code> with two lists produces?	A cross product: use <code>expand_kwargs</code> for pairs
Explain what a deferred task costs in worker slots?	Nothing: the triggerer holds it
Say what happens if the triggerer is not running?	Deferred tasks hang silently
Explain data-aware scheduling in one sentence?	The consumer runs when its datasets are updated
Say why a DAG factory must not query a database?	It runs on every parse
Name what a pool protects that <code>max_active_tasks</code> does not?	An external system, across all DAGs
Say what an SLA miss does <code>not</code> do?	Stop or fail anything
Give the exception that fails without retrying?	<code>AirflowFailException</code>
Name the cheapest high-value test?	The <code>DagBag</code> integrity test
Say what a custom XCom backend makes your problem?	Cleaning up the stored objects



Explain the difference between Cron and CronTriggerTimetable ?	Files after the interval versus at the time
Say why teardown failure should not fail the run?	The output is still valid; alert instead

Senior takes every one of these further, with a security, scale, or ownership dimension.

Who can see and trigger which DAG, and how multi-tenancy works. Secrets backends so credentials never live in the metadata database at all. Scaling the scheduler, the database, and the executor, and knowing which one is your bottleneck. Cost modelling across executors and idle capacity. Upgrade and migration strategy without a maintenance window nobody will grant you. Observability, SLOs, and the metrics that predict an incident. Incident playbooks for a stuck scheduler, a poisoned queue, a bloated metadata database, and a backfill that took down production. Running Airflow as a platform for many teams. Where Airflow stops and a streaming system, a warehouse scheduler, or an in-database tool begins.



PREVIOUS

Beginner · Tips & Tricks

NEXT UP

Mid-level · Interview Review





LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



Brainsmingle space

Free to join this week with code **FREECORETEAM**



MLOps MENA
COMMUNITY

العربية



Join the Community ↗

